‘ **UNIVERSIDAD NACIONAL DE LOJA**

CARRERA DE COMPUTACIÓN

Asignatura: **Estructura de Datos**

Informe Técnico

Avance 4 del Proyecto ABP (Unidad 01)

Implementación de Navegación Bidireccional (ListaDoble),
Pruebas Unitarias de Integración y Protocolo de Entrega EVA

Grupo N.° 03

Integrantes:

Domenica Narváez
Dario Chillogallo
Galo Benitez ’
Yandry Campoverde
Jose Maldonado

Repositorio en GitHub:

`https://github.com/usuario/nombre-repositorio`

Loja, 30 de octubre de 2026

Universidad Nacional de Loja
Carrera en Computación
Facultad de Energía las Industrias y los Recursos Naturales no Renovables



Actividad

Resolución de ejercicios conceptuales mediante la representación gráfica de bloques de memoria (Stack y Heap), direcciones simbólicas y evolución de referencias ante operaciones de instanciación y liberación.Trabajo autónomo colaborativo orientado al diseño de baterías de pruebas unitarias y a la integración del módulo de listas en el repositorio de control de versiones del equipo.

Desarrollo

0.1. Caso de Estudio 1: Shared Lexical Context Leak

```
let ejecutorGlobal = null;

function crearProcesadorResiduos() {
   //Asignacion de 10 MB en el Heap
   let datosMasivos = new Array(10 * 1024 * 1024).fill("DATA_CRITICA");
   let copiaAnterior = ejecutorGlobal;

   //Funcion interna 1: NUNCA se invoca directamente, pero utiliza
   datosMasivos
   let funcionRetenida = function () {
      if (datosMasivos) {
      console.log("Referencia activa a datos masivos");
      }
   };

   //Funcion interna 2: Sobrevive asignandose a la variable global
   ejecutorGlobal = function () {
      console.log("Ejecutando proceso liviano...");
      if (copiaAnterior) {
      copiaAnterior();
      }
   };

// Simulacion de llamadas repetidas
setInterval(() => {
   crearProcesadorResiduos();
   console.log(`Heap usado: ${(process.memoryUsage().heapUsed / 1024 /
   1024).toFixed(2)} MB`);
}, 1000);
}
```

- **Primera ejecucion**

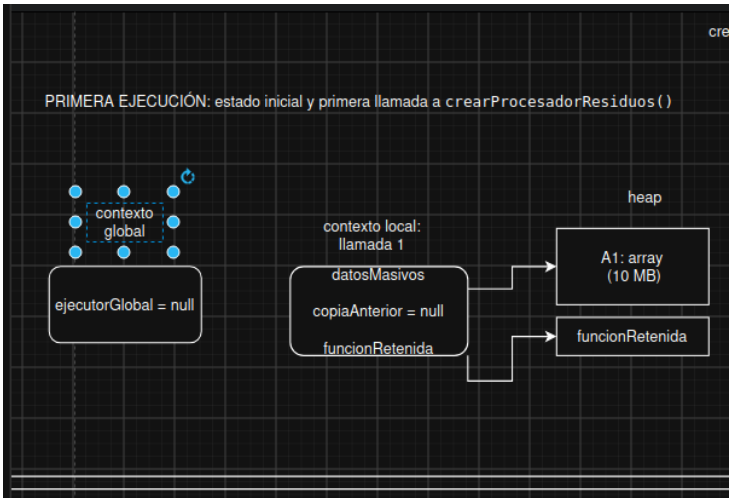


Figura 1: Primera ejecución: estado inicial y primera llamada a `crearProcesadorResiduos()`.

Al llamar por primera vez a `crearProcesadorResiduos()` se crea su contexto local. En el Heap se reservan el arreglo `datosMasivos` (A1, 10 MB) y `funcionRetenida`. Como `ejecutorGlobal` todavía es `null`, `copiaAnterior` también vale `null`.

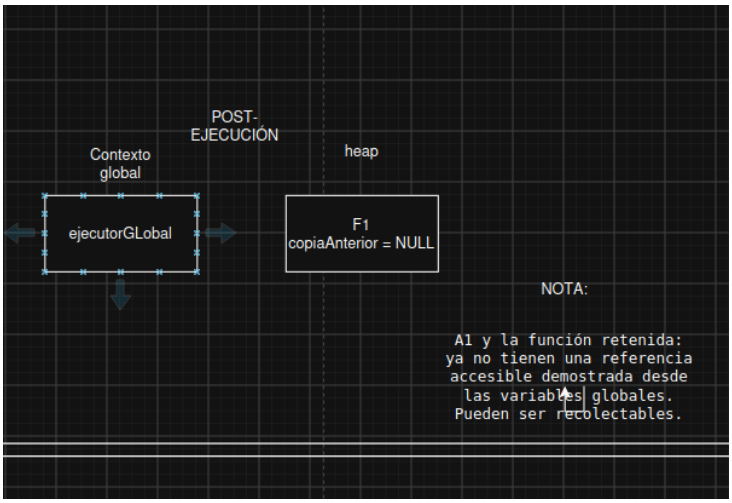


Figura 2: Primera ejecución: estado post-ejecución.

Al terminar la llamada, el contexto local desaparece del Stack y `ejecutorGlobal` queda apuntando a la función F1, cuyo `copiaAnterior` es `null`. Las funciones internas comparten el mismo contexto léxico, que contiene a `datosMasivos`; por eso A1 puede seguir ocupando memoria aunque F1 no lo use.

- **Segunda ejecucion**

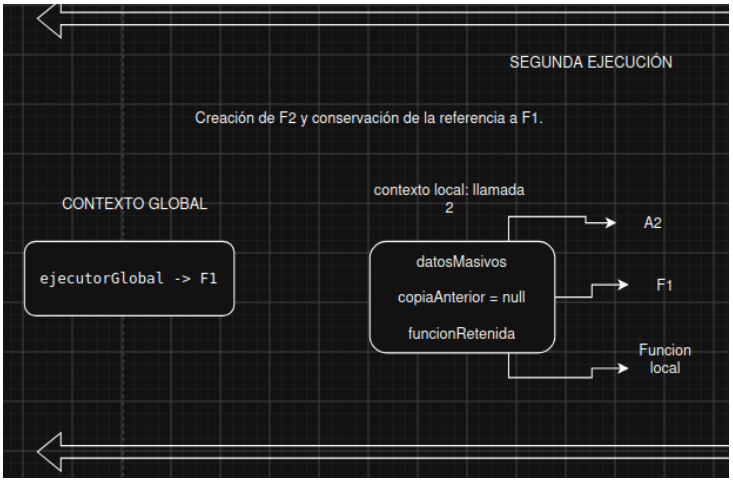


Figura 3: Segunda ejecución: creación de F2 y conservación de la referencia a F1.

En la segunda llamada se crea un nuevo contexto local con un nuevo arreglo (A2, 10 MB) y una nueva función. Esta vez `copiaAnterior` guarda la referencia a F1, que era el valor de `ejecutorGlobal`.

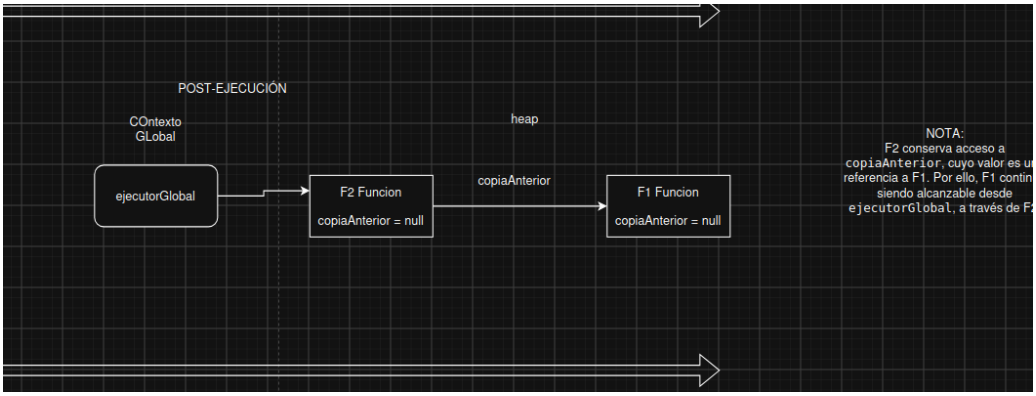


Figura 4: Segunda ejecución: estado post-ejecución.

Al finalizar, `ejecutorGlobal` apunta a F2, y F2 conserva en `copiaAnterior` la referencia a F1. F1 sigue siendo alcanzable desde la variable global a través de F2, por lo que no puede ser liberada.

- **Tercera ejecucion**

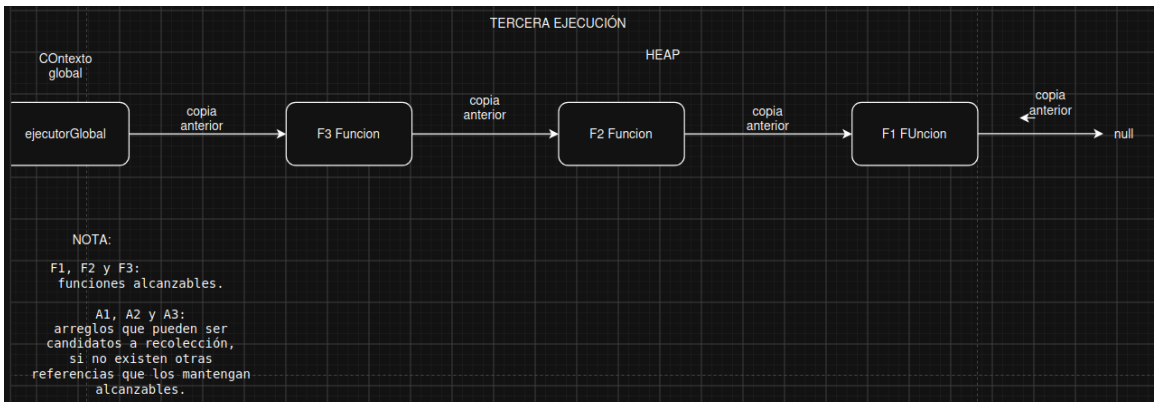


Figura 5: Tercera ejecución: cadena de funciones alcanzables desde `ejecutorGlobal`.

Con la tercera llamada se forma una cadena `ejecutorGlobal` → F3 → F2 → F1 → `null`. Cada nueva ejecución añade un eslabón y el uso de memoria crece de forma continua, que es el patrón de una fuga de memoria.

0.2. Caso de Estudio 2: Grafo Circular y Referencias Débiles

```
// Caso de Estudio 2: Grafo Circular y Referencias Debiles
class NodoGrafo {
    constructor(id) {
        this.id = id;
        this.vecinos = [];
    }

    conectarCon(nodo) {
        this.vecinos.push(nodo);
    }
}

function construirGrafoCiclico() {
    let nodoA = new NodoGrafo("A");
    let nodoB = new NodoGrafo("B");
    let nodoC = new NodoGrafo("C");

    // Conexion circular bidireccional
    nodoA.conectarCon(nodoB);
    nodoB.conectarCon(nodoC);
    nodoC.conectarCon(nodoA);

    // Registro en un gestor centralizado
    gestorCache.set(nodoA.id, nodoA);

    return nodoA.id;
}

const gestorCache = new Map(); // Map o WeakMap

let idRaiz = construirGrafoCiclico();
```

```
32
33 // El estudiante debe analizar que sucede al ejecutar:
34 // gestorCache.delete(idRaiz);
```

- **Estado A:** dentro de `construirGrafoCiclico`

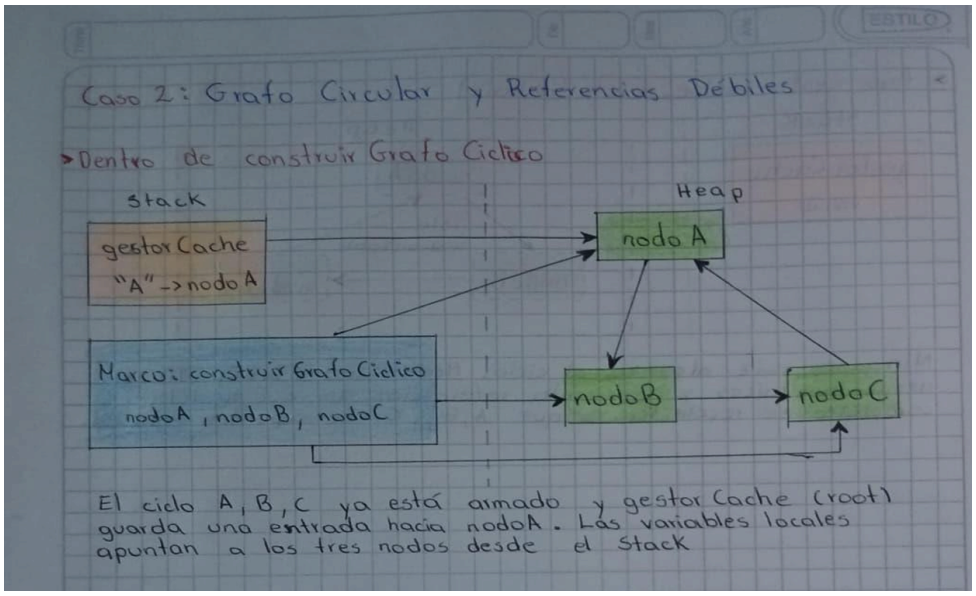


Figura 6: Estado A: dentro de `construirGrafoCiclico`.

Se crean los tres nodos y se conectan en círculo: A apunta a B, B a C y C a A. Las variables locales del marco de la función apuntan a los tres nodos en el Heap, y además `gestorCache` guarda la entrada `."A"` → `nodoA`. En este momento todo está vivo.

- **Estado B:** tras retornar la función

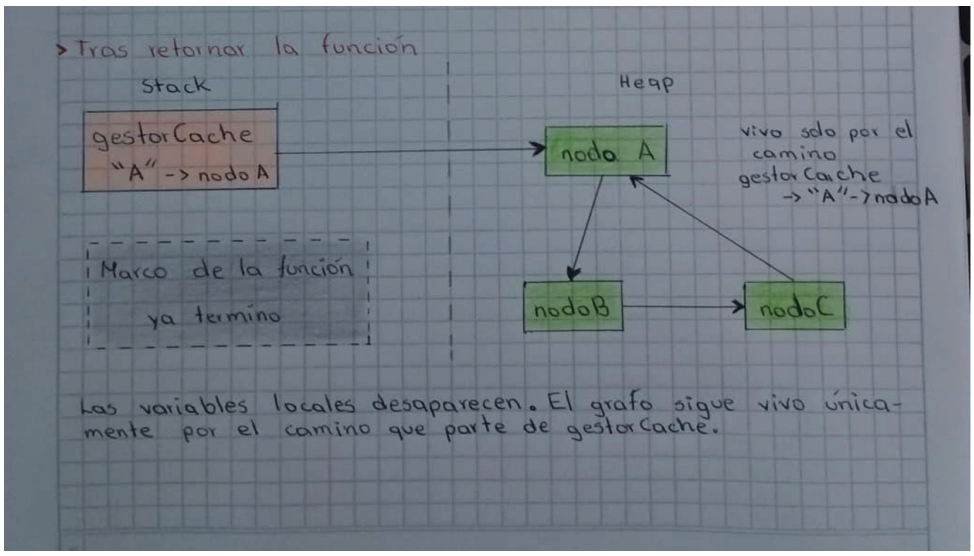


Figura 7: Estado B: tras retornar la función.

La función termina y su marco desaparece del Stack junto con `nodoA`, `nodoB` y `nodoC`. Los nodos siguen vivos solo por el camino `gestorCache` → `."A"` → `nodoA`, desde donde se llega a B y C por las flechas del ciclo.

- **Estado C:** tras `gestorCache.delete(idRaiz)`

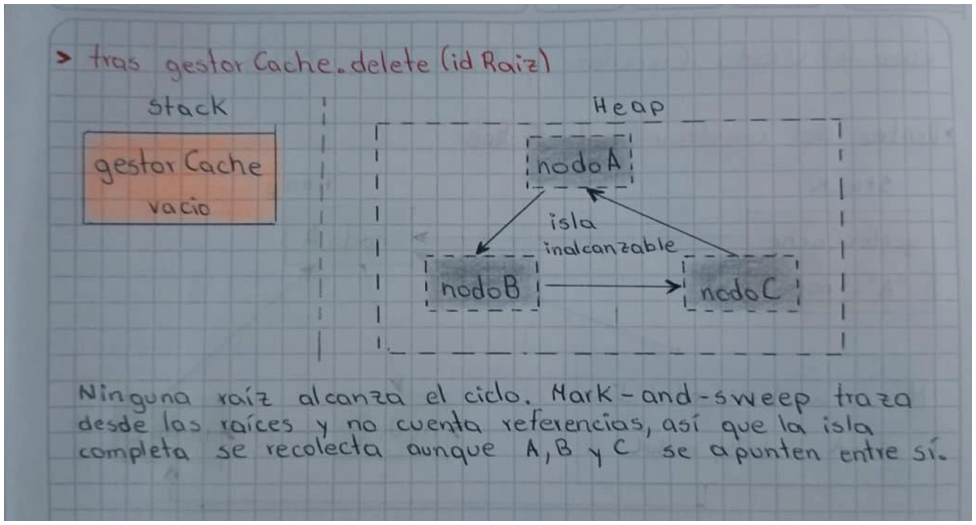


Figura 8: Estado C: tras `gestorCache.delete(idRaiz)`.

Al borrar la entrada del Map ninguna raíz llega al grafo, y los tres nodos quedan como una isla inalcanzable aunque se sigan apuntando entre sí. El recolector usa *mark-and-sweep*: parte de las raíces, marca lo que alcanza y libera el resto, así que el ciclo se recolecta completo.

0.3. Caso de Estudio 3: Event Loop Queue & Retención por Callbacks

```
const EventEmitter = require('events');
const emisorGlobal = new EventEmitter();

class GestorSesionUsuario {
    constructor(idUsuario) {
        this.idUsuario = idUsuario;
        // Reserva de 5 MB de datos por sesion
        this.bufferSesion = new Uint8Array(5 * 1024 * 1024);

        // Suscripcion al emisor global
        this.oyenteEvento = () => {
            this.actualizarActividad();
        };

        emisorGlobal.on('pingSistema', this.oyenteEvento);
    }

    actualizarActividad() {
        console.log("Sesion ${this.idUsuario} activa.");
    }
```

```
21
22      cerrarSesion() {
23          console.log(`Cerrando sesion de ${this.idUsuario}`);
24          // NOTA: El desarrollador olvido desregistrar el evento en
    emisorGlobal
25      }
26      }
27
28      function simularTrafico() {
29      for (let i = 0; i < 50; i++) {
30          let usuario = new GestorSesionUsuario(`USR_${i}`);
31          usuario.cerrarSesion();
32          // La variable local `usuario` sale de alcance (Stack) al
    terminar la iteracion
33      }
34  }
35
36  simularTrafico();
37  // Forzar la ejecucion del evento global
38  emisorGlobal.emit('pingSistema');
```

- **Ejecucion 1:** creación de la primera sesión

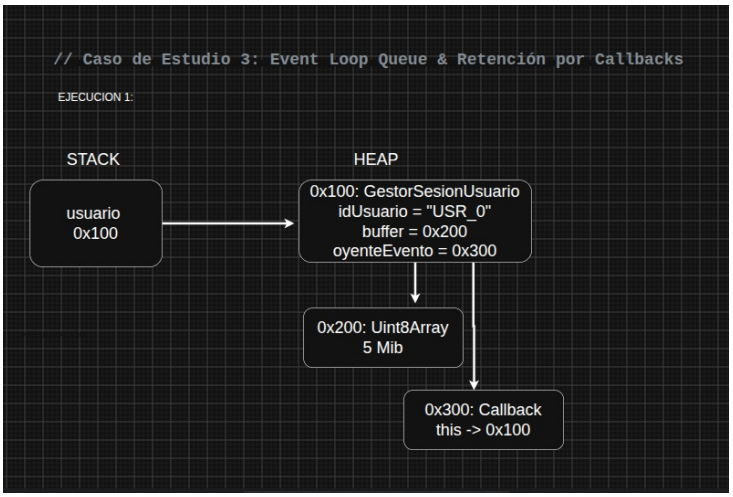


Figura 9: Ejecución 1: primera sesión (`USR_0`) en Stack y Heap.

Al crear el primer usuario (`USR_0`) se instancia un `GestorSesionUsuario` en el Heap (0x100), al que apunta la variable local `usuario`. El objeto contiene un buffer de 5 MiB (0x200) y un callback (0x300) que conserva una referencia al objeto mediante `this` y se registra en `emisorGlobal`.

- **Ejecucion 2:** 50 sesiones creadas

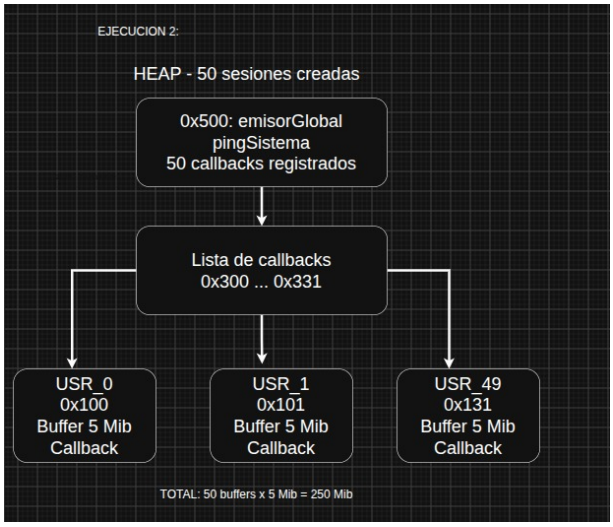


Figura 10: Ejecución 2: 50 sesiones creadas y registradas en `emisorGlobal`.

Durante `simularTrafico()` se crean 50 sesiones, cada una con su buffer de 5 MiB y su callback registrado en el evento `pingSistema`. En total hay 50 buffers $\times$ 5 MiB = 250 MiB en el Heap.

- **Ejecucion 3:** fin de `simularTrafico()`

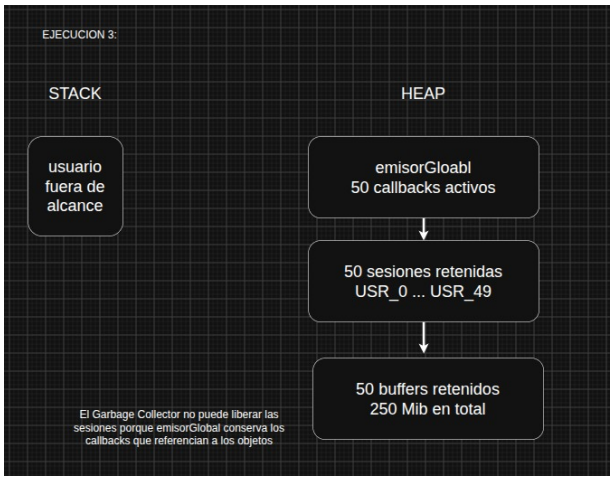


Figura 11: Ejecución 3: sesiones retenidas tras salir de alcance la variable `usuario`.

Al terminar el ciclo la variable `usuario` sale de alcance, pero `cerrarSesion()` solo escribe un mensaje y no desregistra el evento. `emisorGlobal` conserva los 50 callbacks, que referencian a sus objetos, por lo que el Garbage Collector no puede liberar las sesiones ni sus 250 MiB.

- **Ejecucion 4:** emisión de `pingSistema`

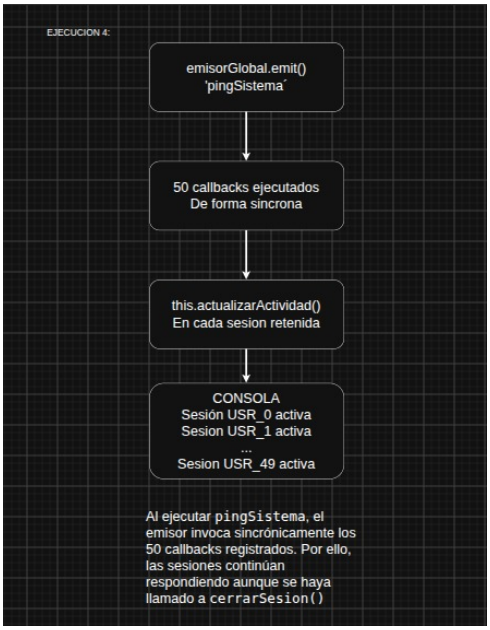


Figura 12: Ejecución 4: `emisorGlobal.emit('pingSistema')`.

Al ejecutar `emisorGlobal.emit('pingSistema')` se invocan de forma síncrona los 50 callbacks, y cada uno llama a `actualizarActividad()` sobre su sesión. Esto demuestra que las sesiones "cerradas" siguen en memoria y respondiendo al evento.

Conclusiones

- C01: En el Caso 1, las funciones internas comparten un mismo contexto léxico, por lo que una función global puede retener datos pesados que no usa. Al encadenarse mediante `copiaAnterior`, cada llamada añade 10 MB que no se liberan, lo que produce una fuga de memoria progresiva.
- C02: En el Caso 2, el recolector de basura (*mark-and-sweep*) decide qué liberar según la alcanzabilidad desde las raíces y no según el número de referencias. Por eso un grafo circular se recolecta completo cuando deja de ser alcanzable, aunque sus nodos se apunten entre sí.
- C03: En el Caso 3, un callback registrado en un emisor global mantiene vivo al objeto que lo creó mediante `this`. Mientras no se desregistre el oyente, las 50 sesiones y sus 250 MiB siguen alcanzables aunque se llame a `cerrarSesion()`.
- C04: En conjunto, los tres casos muestran que las fugas de memoria en JavaScript suelen deberse a referencias olvidadas desde raíces globales (variables, mapas o emisores de eventos). Liberar recursos de forma explícita, como desregistrar eventos con `removeListener` o eliminar entradas de cachés, y diagramar el Stack y el Heap permite detectar estos problemas.